UNIVERSITY OF CALIFORNIA, MERCED

CSE179: Parallel Computing

Feb 4, 9 and 11

Dong Li

Spring 2026

UCMERCED

www.ucmerced.edu

Lecture topics: OpenMP

- Basic Concept: Parallel Region
- The For Construct
- The Single Construct
- The Task Construct
- Scoping: Managing the Data Environment
- Synchronization: the Critical and Reduction Constructs
- More Components of OpenMP

What is OpenMP?

- Defacto standard API for writing shared memory parallel applications in C, C++, and Fortran
- OpenMP supports multi-platform programming
  - On most processor architectures
  - With most OS (Solaris, AIX, HP-UX, Linux, Mac OS X, and Windows)
- OpenMP API consists of:
  - Compiler directives
  - Runtime subroutines/functions
  - Environment variables

Hello World

```
PROGRAM HELLO
!$OMP PARALLEL
PRINT *,"Hello World"
!$ OMP END PARALLEL
STOP
END
```

```
#include <iostream>
#include "omp.h"
int main() {
#pragma omp parallel
  {
    std::cout <<  "Hello World\n"
  }
  return 0;
}
```

```
intel: ifort -openmp -o hi.x hello.f
pgi:   pgfortran -mp -o hi.x hello.f
gnu:  gfortran -fopenmp -o hi.x hello.f
```

```
intel: icc -openmp -o hi.x hello.f
pgi:   pgcpp -mp  -o hi.x hello.f
gnu:  g++ -fopenmp -o hi.x hello.f
```

```
Export OMP_NUM_THREADS=4
./hi.x
```

Hello World

```
PROGRAM HELLO
!$OMP PARALLEL
PRINT *,"Hello World"
!$ OMP END PARALLEL
STOP
END
```

```
#include <iostream>
#include "omp.h"
int main() {
#pragma omp parallel
  {
    std::cout <<  "Hello World\n"
  }
  return 0;
}
```

OMP COMPILER DIRECTIVES

```
intel: ifort -openmp -o hi.x hello.f
pgi:   pgfortran -mp -o hi.x hello.f
gnu:  gfortran -fopenmp -o hi.x hello.f
```

```
intel: icc -openmp -o hi.x hello.f
pgi:   pgcpp -mp  -o hi.x hello.f
gnu:  g++ -fopenmp -o hi.x hello.f
```

```
Export OMP_NUM_THREADS=4
./hi.x
```

OMP ENVIRONMENTAL VARIABLE

NOTE: example hello.f90

Fortran, C/C++

- C/C++ directive format is:

```
#pragma omp parallel [clauses]
{
   …
}
```

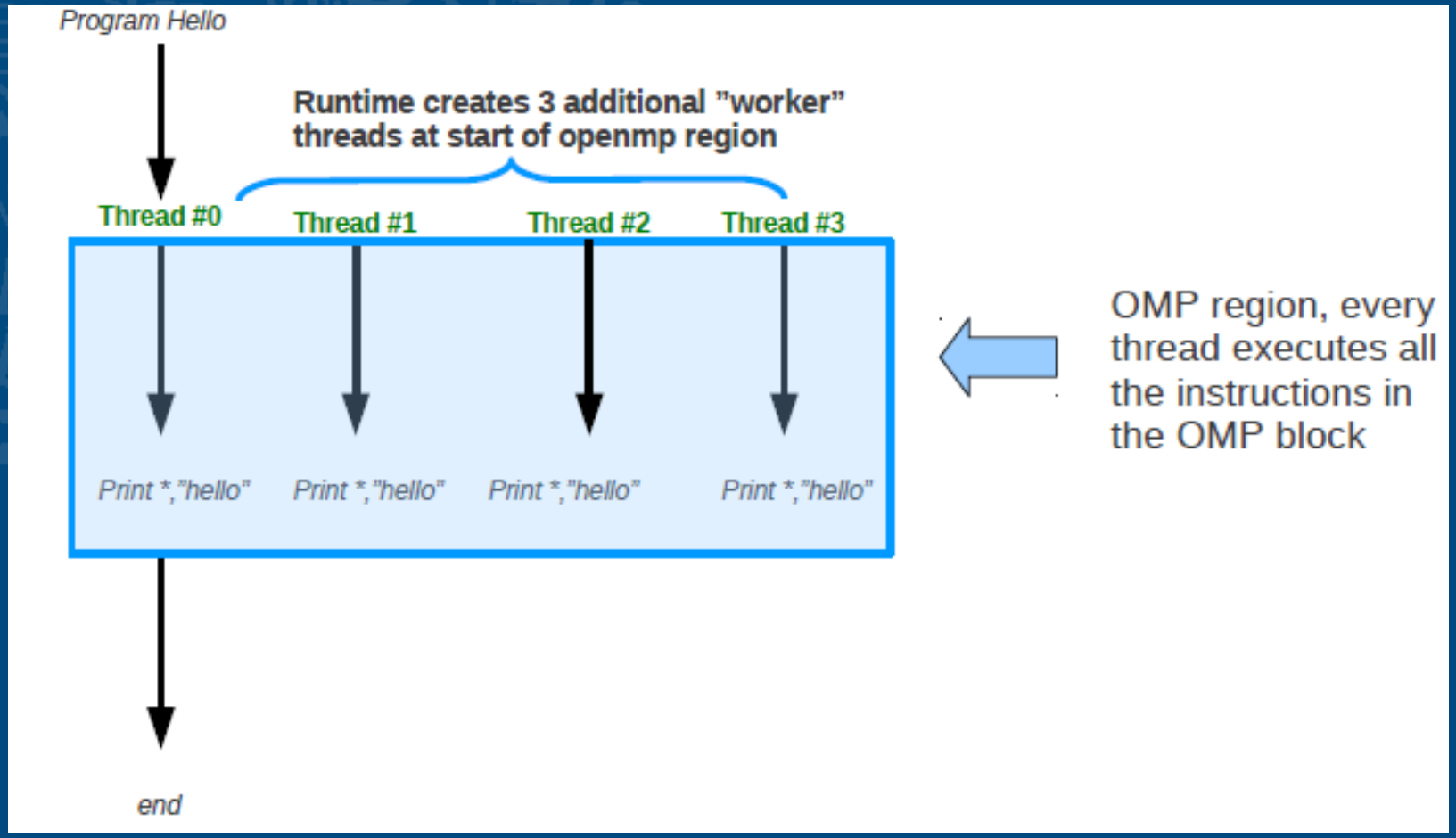
- Fortran directive format

```
!$ OMP PARALLEL [clauses]
   :
!$OMP END PARALLEL
```

All OpenMP directives follow this format

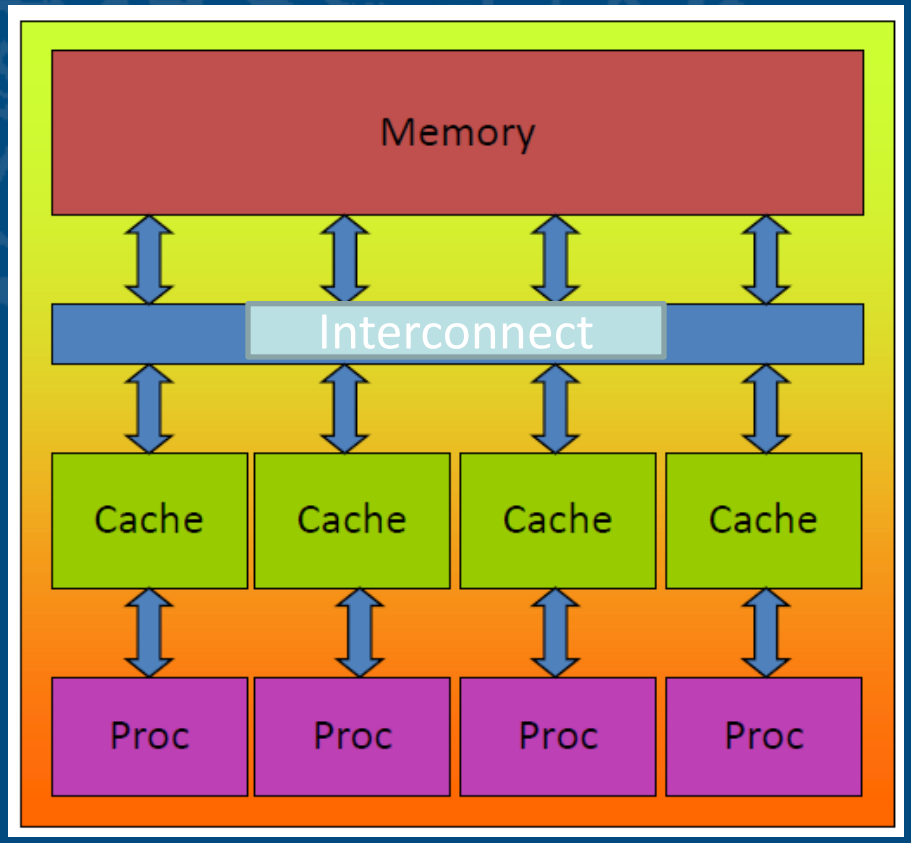
UCMERCED

At runtime


Program Hello
Runtime creates 3 additional "worker" threads at start of openmp region
Thread #0
Thread #1
Thread #2
Thread #3
Print *,"hello"
Print *,"hello"
Print *,"hello"
Print *,"hello"
OMP region, every thread executes all the instructions in the OMP block
end

OpenMP's machine model

- OpenMP: Shared-Memory Parallel Programming Model

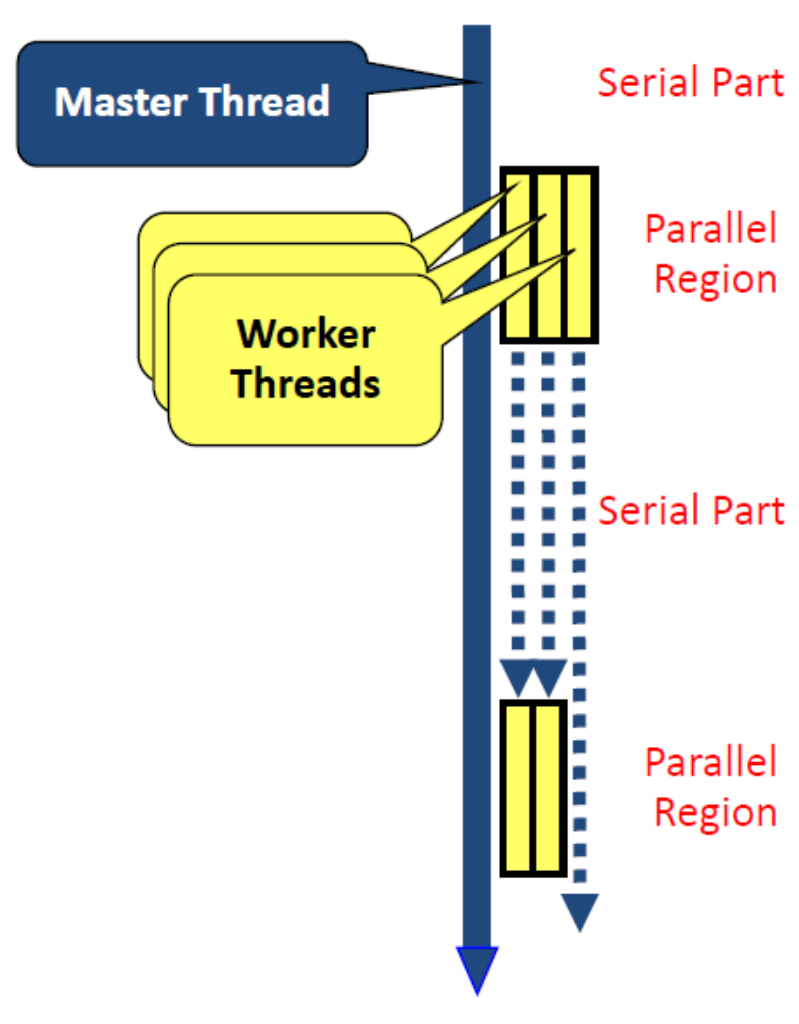


- All processors/cores access a shared main memory
- Parallelization in OpenMP employs multiple threads

OpenMP execution model

- OpenMP programs start with just one thread: The *Master*

- *Worker* threads are spawned at *Parallel Regions*; together with the Master they form the *Team* of threads

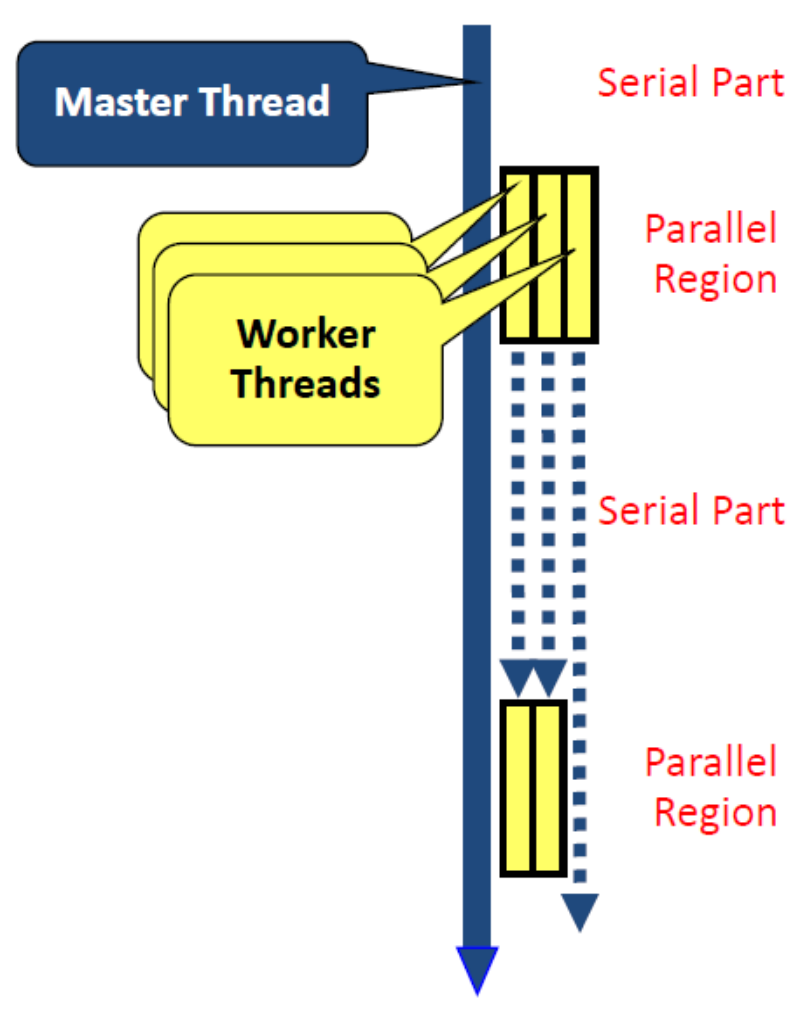


OpenMP execution model

- In between Parallel Regions the Worker threads are put to sleep. The OpenMP *Runtime* takes care of all thread management work

- Allows for an incremental parallelization!

- Concept: Fork-join



Fork/Join model

- At start of parallel region master creates team of parallel "worker" threads (FORK)
- Statements in parallel block are executed in parallel by every thread
- At the end of parallel region, all threads synchronize, and join master thread (JOIN)
  - Implicit barrier. Will discuss synchronization later

OpenMP Threads versus Cores

--- what are threads, cores and how do they relate?

- Thread is independent sequence of execution of program code
  - Block of code with one entry and one exit
  - For our purposes a more abstract concept
- Unrelated to Cores/CPUs
- OpenMP threads are mapped onto physical cores
- Possible to map more than 1 thread on a core
- In practice best to have one-to-one mapping.

More About OpenMP Threads

- Number of openMP threads can be set using
  - Environmental variable **OMP_NUM_THREADS**
  - Runtime function **omp_set_num_threads(n)**
- Other useful function to get information about threads:
  - Runtime function **omp_get_num_threads()**
    - Returns number of threads in parallel region
    - Returns 1 if called outside parallel region
  - Runtime function **omp_get_thread_num()**
    - Returns id of thread in team
    - Value between [0,n-1] // where n = #threads
    - Master thread always has id 0

Parallel Region

Parallel Region and Structured Blocks

- The parallelism has to be expressed explicitly

C/C++

```
#pragma omp parallel
{
    ...
    structured block
    ...
}
```

Fortran

```
!$omp parallel
    ...
    structured block
    ...
$!omp end parallel
```

- Structured block
  - Exactly one entry point at the top
  - Exactly one exit point at the bottom
  - Branching in or out is not allowed
  - Terminating the program is allowed (abort / exit)

UCMERCED

Parallel Region and Structured Blocks

- The parallelism has to be expressed explicitly

C/C++

```
#pragma omp parallel
{
    ...
    structured block
    ...
}
```

Fortran

```
!$omp parallel
    ...
    structured block
    ...
$!omp end parallel
```

- Specification of number of threads
  - Environment variable:
    - OMP_NUM_THREADS=…
  - via *num_threads* clause

    add *num_threads (num)* to the parallel construct
  - From within a shell, one-time adjustment of the number of threads:
    - OMP_NUM_THREADS=4 ./program

UCMERCED

For Construct

UCMERCED

For Worksharing

- If only the *parallel* construct is used, each thread executes the Structured Block

- OpenMP's most common Worksharing construct: *for*

For Worksharing

C/C++

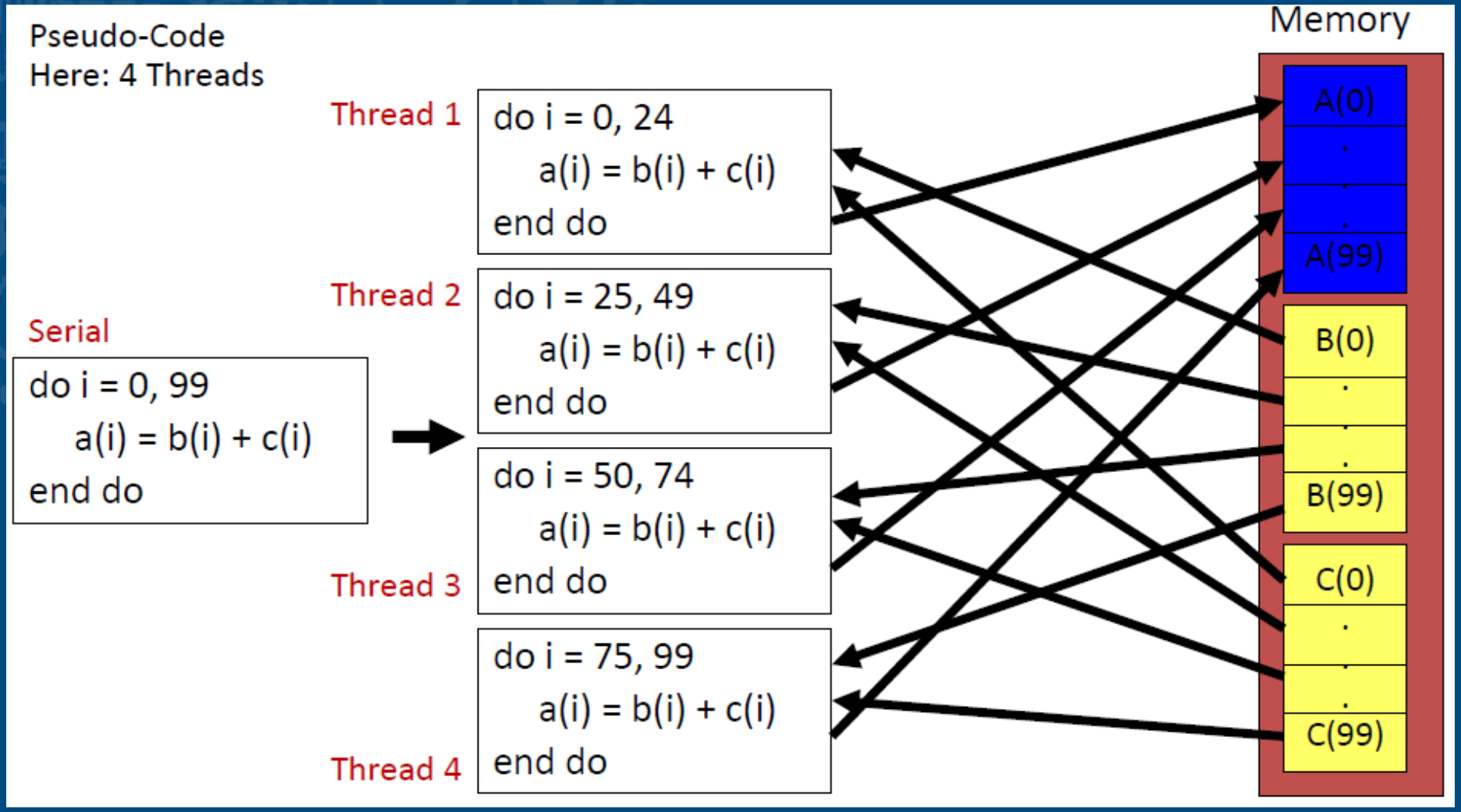
```
int i;
double a[N], b[N], c[N];
#pragma omp parallel for
for (i = 0; i < N; i++)
{
   a[i] = b[i] + c[i];
}
```

Fortran

```
INTEGER :: i
INTEGER, DIMENSION(N) :: a,b,c
!$omp parallel do
DO i = 1, N
   a[i] = b[i] + c[i];
END DO
```

- Distribution of loop iterations over all threads in a Team
- Scheduling of the distribution can be influenced
- Loops often account for most of a program‘s runtime!

Worksharing illustrated



OpenMP Scheduling

UCMERCED

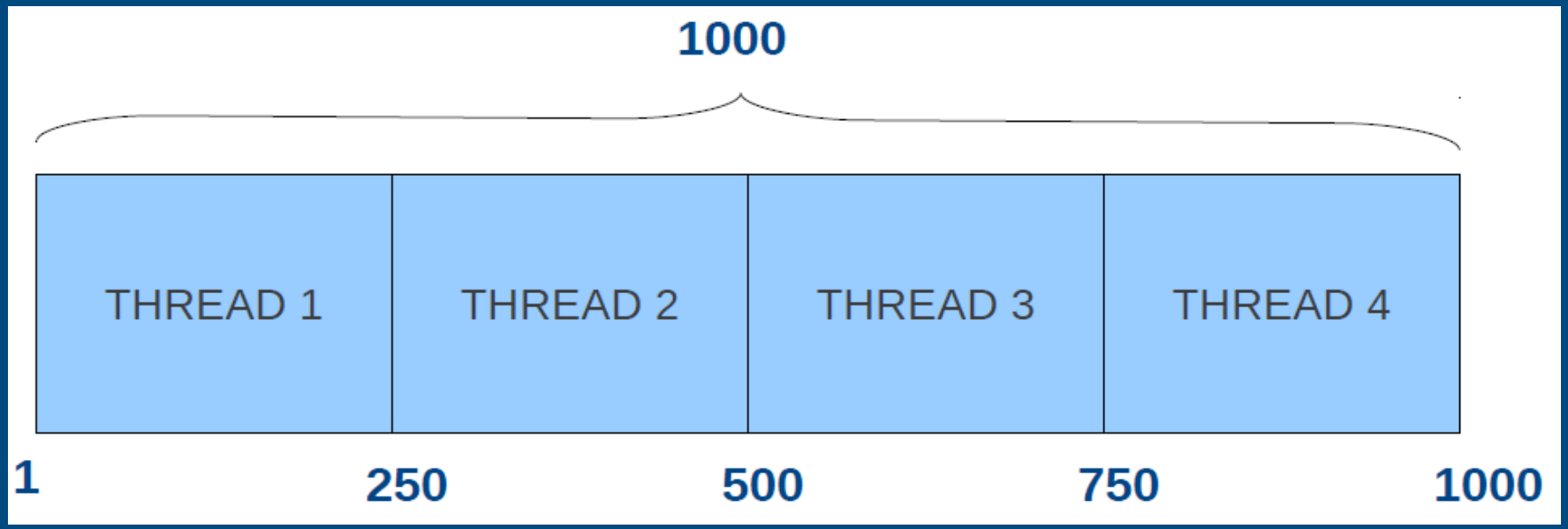
How OMP schedule iterations?

- The OpenMP standard does not specify how a loop should be partitioned
- Static schedule
- Dynamic schedule

Static schedule

- Split the loop in *N*/*p* (*N* #iterations, *p* #threads) chunks (with chunk size N/p)

- For example, suppose we have a loop with 1000 iterations and 4 omp threads. The loop is partitioned as follows.

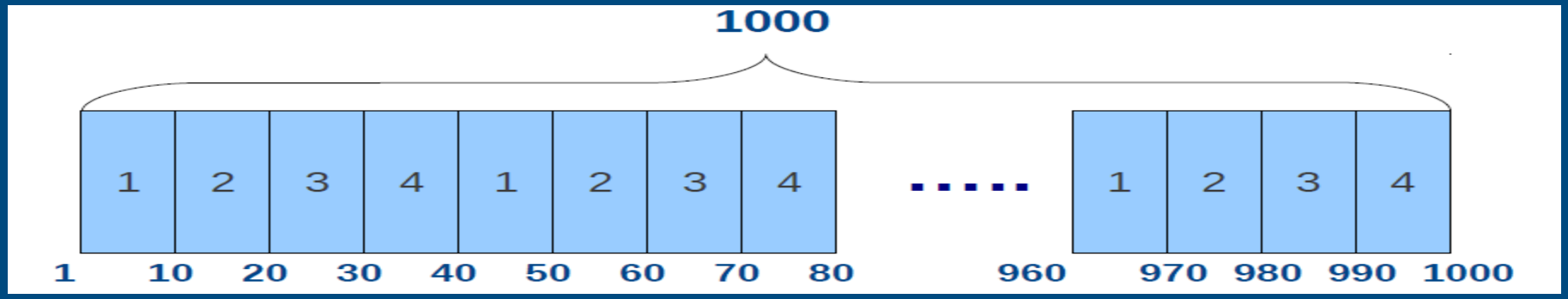


Static schedule

- To explicitly tell the compiler to use a static schedule (or a different schedule as we will see later) OpenMP provides the SCHEDULE clause

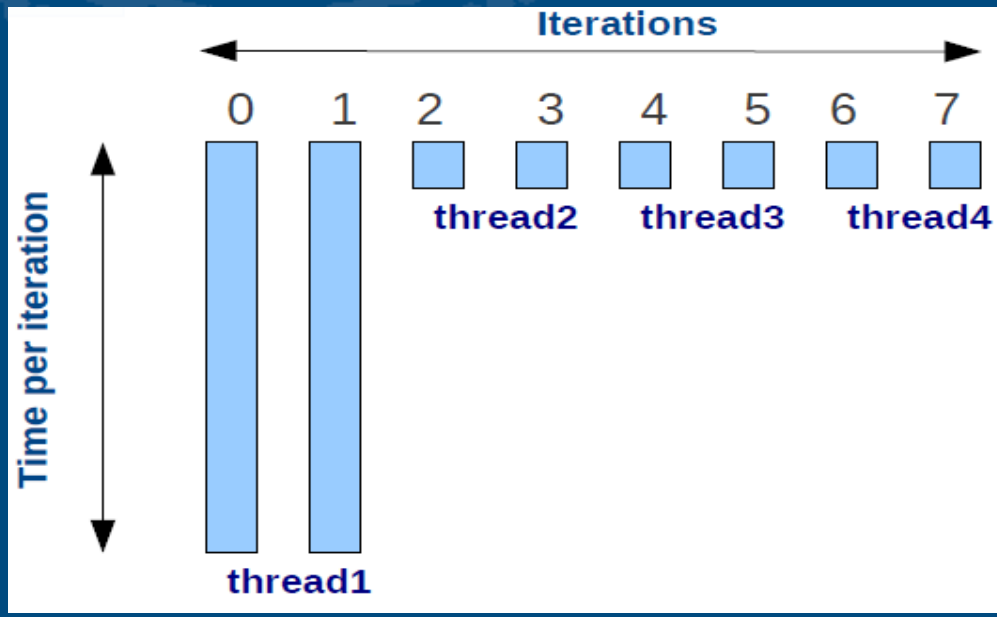
#pragma omp for *[clause …]* schedule (static, *chunksize*)

For example, *suppose we set the chunk size n=10*



Problems with static schedule

- With static scheduling the number of iterations is evenly distributed among all openmp threads (i.e. Every thread will be assigned similar number of iterations).
  - This is not always the best way to partition.



This is called load imbalance. In this case threads 2,3, and 4 will be waiting very long for thread 1 to finish

Dynamic schedule

- With a dynamic schedule new chunks are assigned to threads when they come available. OpenMP provides two dynamic schedules
  - **#pragma omp for *[clause …]* schedule (dynamic, *chunksize*)**

    Loop iterations are divided into pieces of size chunk. When a thread finishes one chunk, it is dynamically assigned another
  - **#pragma omp for *[clause …]* schedule (guided, *chunksize*)**

    Similar to DYNAMIC but chunk size is relative to number of iterations left

Dynamic schedule

- Guided scheduling
  - The size of the initial chunk is proportional to:
    - number_of_iterations / number_of_threads
  - Subsequent chunks are proportional to
    - number_of_iterations_remaining / number_of_threads
  - The chunk parameter defines the minimum chunk size.
    - The default chunk size is 1

- Keep in mind: although Dynamic scheduling might be the prefered choice to prevent load imbalance in some situations, there is a significant overhead involved compared to static scheduling

Single Construct

UCMERCED

Single Construct

C/C++

```
#pragma omp single [clause]
... structured block ...
```

Fortran

```
!$omp single [clause]
... structured block ...
!$omp end single
```

- The single construct specifies that the enclosed structured block is executed by only one thread of the team
  - It is up to the runtime which thread that is
- Useful for:
  - I/O
  - Memory allocation and deallocation, etc. (in general: setup work)

Data Dependencies

UCMERCED

Data dependencies (1)

- Not all loops can be parallelized. Before adding OpenMP directives need to check for any dependencies

- We categorize three types of dependencies
  - Flow dependence: Read after Write (RAW)
  - Anti-dependence: Write after Read (WAR)
  - Output dependence (Write after Write (WAW)

FLOW	ANTI	OUTPUT
X = 21 *PRINT *, X*	*PRINT *, X* *X = 21*	*X = 21* *X = 21*

Data dependencies (2)

- For our purpose (openMP parallel loops) we only care about loop carried dependencies (dependencies between instructions in different iterations of the loop)

- Let's find the dependencies in the following loop

```
S1: DO I=1,10
S2:     B(i) = temp
S3:     A(i+1) = B(i+1)
S4:     temp = A(i)
S5: ENDDO
```

Data dependencies (2)

- For our purpose (openMP parallel loops) we only care about loop carried dependencies (dependencies between instructions in different iterations of the loop)

- Let's find the dependencies in the following loop

```
S1: DO I=1,10
S2:    B(i) = temp
S3:    A(i+1) = B(i+1)
S4:    temp = A(i)
S5: ENDDO
```

1: S3 → S2 anti (B)

Data dependencies (2)

- For our purpose (openMP parallel loops) we only care about loop carried dependencies (dependencies between instructions in different iterations of the loop)

- Let's find the dependencies in the following loop

```
S1: DO I=1,10
S2:    B(i) = temp
S3:    A(i+1) = B(i+1)
S4:    temp = A(i)
S5: ENDDO
```

1: S3 → S2 anti (B)
2: S3 → S4 flow (A)

Data dependencies (2)

- For our purpose (openMP parallel loops) we only care about loop carried dependencies (dependencies between instructions in different iterations of the loop)

- Let's find the dependencies in the following loop

What are the dependencies in the following loop?

```
S1: DO I=1,10
S2:    B(i) = temp
S3:    A(i+1) = B(i+1)
S4:    temp = A(i)
S5: ENDDO
```

1: S3 → S2 anti (B)
2: S3 → S4 flow (A)
3: S4 → S2 flow (temp)

Data dependencies (2)

- For our purpose (openMP parallel loops) we only care about loop carried dependencies (dependencies between instructions in different iterations of the loop)

- Let's find the dependencies in the following loop

What are the dependencies in the following loop?

```
S1: DO I=1,10
S2:     B(i) = temp
S3:     A(i+1) = B(i+1)
S4:     temp = A(i)
S5: ENDDO
```

1: S3 → S2 anti (B)
2: S3 → S4 flow (A)
3: S4 → S2 flow (temp)
4: S4 → S4 output (temp)

Data dependencies (2)

- Let's find the dependencies in the following loop

What are the dependencies in the following loop?

```
S1: DO I=1,10
S2:    B(i) = temp
S3:    A(i+1) = B(i+1)
S4:    temp = A(i)
S5: ENDDO
```

1: S3 → S2 anti (B)
2: S3 → S4 flow (A)
3: S4 → S2 flow (temp)
4: S4 → S4 output (temp)

- Sometimes it helps to ”unroll” part of the loop to see loop carried dependencies more clear

```
S2: B(1) = temp
S3: A(2) = B(2)
S4: temp = A(1)

S2: B(2) = temp
S3: A(3) = B(3)
S4: temp = A(2)
```

UCMERCED

Data dependencies (2)

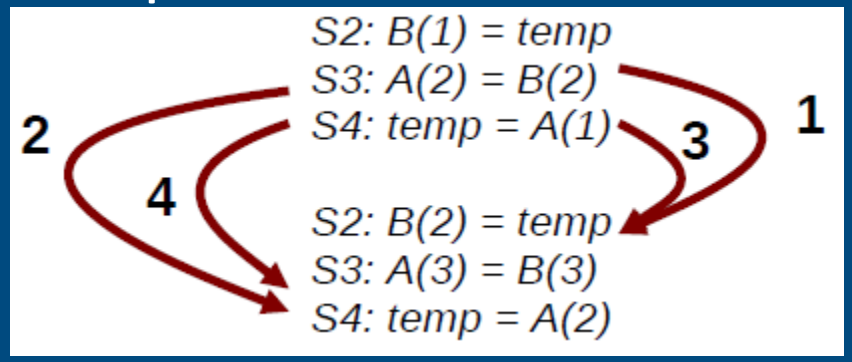
- Let's find the dependencies in the following loop

What are the dependencies in the following loop?

```
S1: DO I=1,10
S2:    B(i) = temp
S3:    A(i+1) = B(i+1)
S4:    temp = A(i)
S5: ENDDO
```

1: S3 → S2 anti (B)
2: S3 → S4 flow (A)
3: S4 → S2 flow (temp)
4: S4 → S4 output (temp)

- Sometimes it helps to ”unroll” part of the loop to see loop carried dependencies more clear



Data Dependencies (3)

- Loop carried anti- and output dependencies are not true dependencies (re-use of the same name) and in many cases can be resolved relatively easily

- Flow dependencies are true dependencies (there is a flow from definition to its use) and in many cases cannot be removed easily. Might require rewriting the algorithm (if possible)

Resolving Anti/Output Deps

- Use PRIVATE clause:
- Rename variables (if possible):

Example: in-place left shift

```
DO i=1,n-1
  A(i)=A(i+1)
ENDDO
```

→

```
DO i=1,n-1
   ANEW(i) = A(i+1)
ENDDO
```

→

```
!$OMP PARALLEL DO
DO i=1,n-1
   ANEW(i) = A(i+1)
ENDDO
!$OMP END PARALLEL DO
```

If has to be in-place can do it in two steps:

```
!$OMP PARALLEL
!$OMP DO
   T(i) = A(i+1)
!$OMP END DO
!$OMP DO
    A(i) = T(i)
!$OMP END DO
!$OMP END PARALLEL
```

UCMERCED

Case Study: Removing Flow Deps

$Y = \text{prefix}(X) \rightarrow Y(1) = X(1);\ Y(i) = Y(i-1) + X(i)$

SEQUENTIAL

```
Y[1] = X[1]
DO i=2,n,1
   Y[i] = Y[i-1] + X[i]
ENDDO
```

Case Study: Removing Flow Deps

Y = prefix(X) → Y(1) = X(1); Y(i) = Y(i-1) + X(i)

SEQUENTIAL

```
Y[1] = X[1]
DO i=2,n,1
   Y[i] = Y[i-1] + X[i]
ENDDO
```

PARALLEL

```
Y[1] = X[1]
!$OMP PARALLEL DO
DO i=2,n,1
   Y[i] = Y[i-1] + X[i]
ENDDO
!$OMP END PARALLEL DO
```

Case Study: Removing Flow Deps

$Y = \text{prefix}(X) \rightarrow Y(1) = X(1);\ Y(i) = Y(i-1) + X(i)$

X			
1	1	1	1

→

Y			
1	2	3	4

SEQUENTIAL

```
Y[1] = X[1]
DO i=2,n,1
   Y[i] = Y[i-1] + X[i]
ENDDO
```

PARALLEL

```
Y[1] = X[1]
!$OMP PARALLEL DO
DO i=2,n,1
   Y[i] = Y[i-1] + X[i]
ENDDO
!$OMP END PARALLEL DO
```

WHY?

Case Study: Removing Flow Deps

REWRITE ALGORITHM

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

STEP 1: split X among threads; every thread computes its own (partial) prefix sum

1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4

STEP 2: create array T → T[1]=0, T[i] = X[(length/threads)*(i-1)], perform simple prefix sum on T

(will collects last element from every thread (except last) and perform simple prefix sum)

0	4	4	4

⇨

0	4	8	12

STEP 3: every thread adds T[theadid] to all its element

+0				+4				+8				+12			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

STEP 4: Finished; we rewrote prefex sum by removing dependencies.

Prefix Sum Implementation

- Three separate steps
- Steps 1 and 3 can be done in parallel
- Step 2 has to be done sequential
- Step 1 has to be performed before step 2
- Step 2 has to be performed before step 3

Case Study: Removing Flow Deps

- This case study showed an example of an algorithm with real (flow) dependencies

- Sometimes we can rewrite algorightm to run parallel
- Most of the time this is not trivial
- Speedup much less impressive (often)

Shared and Private Variables

UCMERCED

Shared and private variables

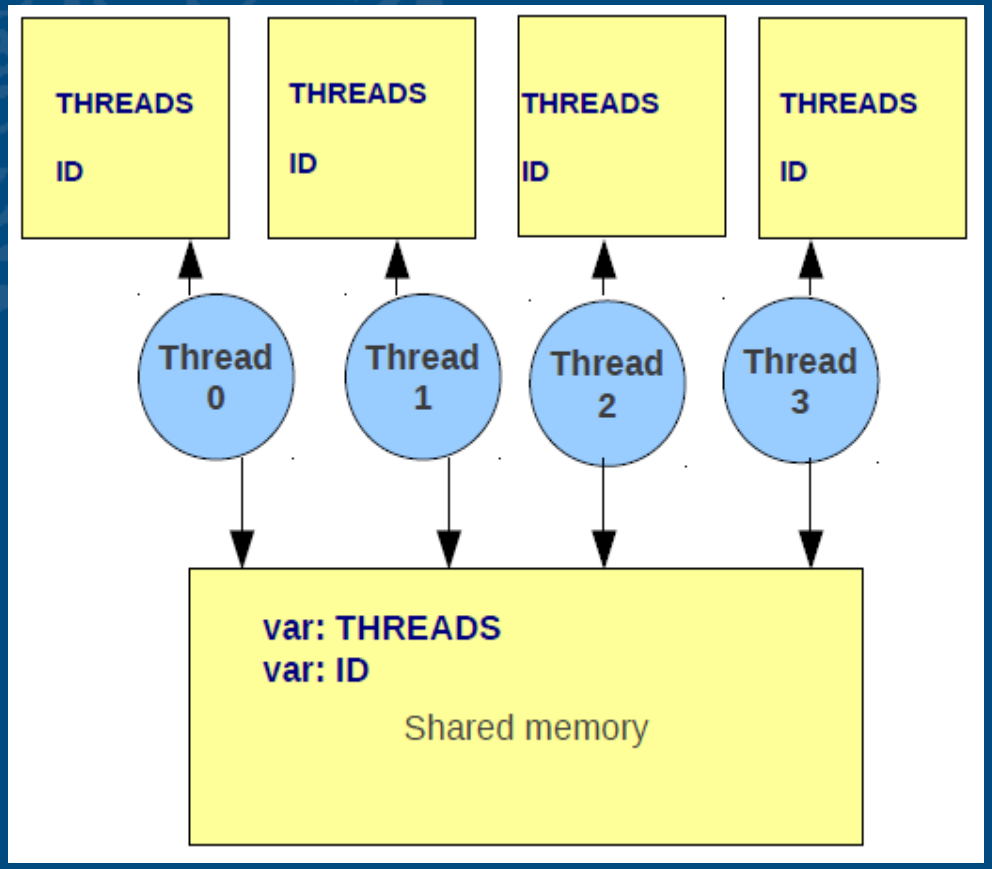
- OpenMP provides a way to declare variables private or shared within an OpenMP block. This is done using the following OpenMP *clauses:*
  - *SHARED (list)*
    - All variables in ***list*** will be considered shared.
    - Every openmp thread has access to all these variables
  - *PRIVATE (list)*
    - Every openmp thread will have its own **"private"** copy of variables in list
    - No other openmp thread has access to this **"private"** copy

By default most variables are considered shared in OpenMP
Exceptions include index variables (Fortran, C/C++) and variables declared inside parallel region (C/C++)

UCMERCED

Revisit Shared Memory Model

The memory is (logically) shared by all the CPU.
There is also private memory for every openmp thread



TIP: practical issue

- OpenMP creates separate data stack for every worker thread to store copies of private variables (master thread uses regular stack)
- Size of these stacks is not defined by OpenMP standards
  - Intel compiler: default stack is 4MB
  - gcc/gfortran: default stack is 2MB
- Behavior of program undefined when stack space exceeded
  - Although most compilers/RT will throw seg fault
- To increase stack size use environment var OMP_STACKSIZE, e.g.
  - export OMP_STACKSIZE=512M
  - export OMP_STACKSIZE=1GB
- To make sure master thread has large enough stack space use
  - ulimit -s command (unix/linux).

Task Construct

UCMERCED

Task construct

C/C++

```
#pragma omp task [clause]
... structured block ...
```

Fortran

```
!$omp task [clause]
... structured block ...
!$omp end task
```

- A task has
  - Code to execute
  - A data environment
  - An assigned thread that executes the code and uses the data
- Two activities
  - Each encountering thread packages a new instance of a task
  - Some thread in the team executes the thread at a later time
- Data scoping clauses
  - sharing (list)
  - private (list) firstprivate (list)
  - default (sharing/none)

UCMERCED

Example – Linked List/1

- Algorithm: Walking through a dynamic linked list
- The number of items in the list is unknown until at runtime
  - It is not readily parallelizable with OpenMP *parallel* construct

```
p = listhead; num_elements=0;
while (p) {
  list_item[num_elements++]=p;
  p=next(p);
}

#pragma omp parallel for
  for (int i=0; i < num_elements; i++)
    process(list_item[i]);
```

- We have to incur the overhead of array construction

UCMERCED

Example – Linked List/2

- Another approach: use the *single nowait* construct inside a parallel region
- All the threads in the team execute the while loop in parallel
- The single directive is used to ensure that only one of the threads in the team actually processes a given list element
- This second approach is unintuitive and inefficient
  - Each thread needs to traverse the whole list and determine for each element whether another thread has already executed the work on that element

```
#pragma omp parallel private (p)
{
  p = listhead;
  while(p) {
    #pragma omp single nowait
      process(p);
    p = next(p);
  }
}
```

UCMERCED

Example – Linked List/3

- Another approach: task-based implementation

- The single construct ensures that only one thread will traverse the list and encounter the task directive

```
#pragma omp parallel
{
  /* a single thread traverses the list */
  #pragma omp single
  {
    p = listhead;
    while(p) {
      /* create a task for each element */
      #pragma omp task
        process(p)
      p=next(p);
    }
  }
}
```

UCMERCED

Tied task and untied task

- A task is *tied* if the code is executed by the same thread from beginning to end
- A task is *untied* if the code can be executed by more than one thread, so that different threads execute different parts of the code
- By default, tasks are tied
- Task can be specified to be untied by using the *untied* clause with the *task* directive

UCMERCED

Task scheduling points

- For tied tasks
  - the point of encountering a task construct
  - the point of encountering a taskwait construct
  - the point of encountering an implicit or explicit barrier
  - the completion point of the task

- For untied tasks
  - The task scheduling points may in principle occur at any point in an untied task region

Scoping

Scoping rules

- Managing the Data Environment is the challenge of OpenMP
- Scoping in OpenMP: dividing variables in *shared* and *private*:
  - *private*-list and *shared*-list on Parallel Region
  - *private*-list and *shared*-list on Worksharing constructs
  - General default is *shared* for Parallel Region, *firstprivate* for Tasks
  - Loop control variables on *for*-constructs are *private*
  - Non-static variables local to Parallel Regions are *private*
  - *private*: A new uninitialized instance is created for each thread
    - firstprivate: Initialization with Master‘s value
    - lastprivate: Value of last loop iteration is written back to Master
  - Static variables are *shared*

UCMERCED

Privatization of Global/Static Variables

- Global / static variables can be privatized with the *threadprivate* directive
  - One instance is created for each thread
    - Before the first parallel region is encountered
    - Instance exists until the program ends
    - Does not work (well) with nested parallel region
  - Based on thread-local storage (TLS)
    - TlsAlloc (Win32-Threads), pthread_key_create (Posix-Threads), keyword __thread (GNU extension)

C/C++

```
static int i;
#pragma omp threadprivate(i)
```

Fortran

```
SAVE INTEGER :: i
!$omp threadprivate(i)
```

Tasks in OpenMP: Data Scoping

- Some rules from *Parallel Regions* apply:
  - Static and Global variables are shared
  - Automatic Storage (local) variables are private

- If shared scoping is not derived by default:
  - Orphaned Task variables are *firstprivate* by default
  - Non-Orphaned Task variables inherit the shared attribute

Synchronization

UCMERCED

Synchronization Overview

- Can all loops be parallelized with for-constructs? No!
  - Simple test: If the results differ when the code is executed backwards, the loop iterations are not independent. BUT: This test alone is not sufficient:

C/C++

```
int i;
double s, a[N];
#pragma omp parallel for
for (i = 0; i < N; i++)
{
    s = s + a[i];
}
```

- Data race: If between two synchronization points at least one thread writes to a memory location from which at least one other thread reads, the result is not deterministic (race condition).

UCMERCED

Synchronization: Critical Region

- A *Critical Region* is executed by all threads, but by only one thread simultaneously (*Mutual Exclusion*)

C/C++

```
#pragma omp critical (name)
{
    ... structured block ...
}
```

- Do you think this solution scales well?

C/C++

```
int i;
double s, a[N];
#pragma omp parallel for
for (i = 0; i < N; i++)
{
#pragma omp critical
    {   s = s + a[i];   }
}
```

UCMERCED

It's your turn: Make It Scale!

```
#pragma omp parallel
{

#pragma omp for
   for (i = 0; i < N; i++)
   {

         s  = s   + a[i];

   }

} // end parallel
```

```
do i = 0, 99
    s = s + a(i)
end do
```

→

```
do i = 0, 24
    s = s + a(i)
end do
```

```
do i = 25, 49
    s = s + a(i)
end do
```

```
do i = 50, 74
    s = s + a(i)
end do
```

```
do i = 75, 99
    s = s + a(i)
end do
```

OMP MASTER

- This Directive ensures that only the master threads excecute instructions in the block
  - There is no implicit barrier so other threads will not wait for master to finish
- What is difference with SINGLE construct?
  - …

OMP Atomic

- This directive is very similar to the *critical* directive on the previous slide
  - Difference is that !$OMP ATOMIC is only used for the update of a memory location.
- Sometimes !$OMP ATOMIC is also referred to as a mini critical section
  - Block consists of only one statement
  - Atomic statement must follow specific syntax

#pragma omp barrier

- It will enforce every thread to wait at the barrier until all threads have reached the barrier
  - probably the most well known synchronization mechanism;

- Explicitly or implicitly, the following omp schemes we discussed before include an implicit barrier:
  - The end of a parallel region (!$OMP END PARALLEL)
  - The end of a parallel for (!$OMP END DO)
  - The end of a “single” program block (!$ OMP END SINGLE)
  - The end of a “critical” program block (!$ OMP END CRITICAL)

NOWAIT Clause

- All threads will synchronize at the end of work sharing construct

- For situations where we don't need or want to synchronize at the end OpenMP provides the NOWAIT clause

Fortran

```
!$OMP DO
    :
!$OMP END DO NOWAIT
```

C/C++

```
#pragma omp for nowait
{
    :
}
```

Reduction

Reductions

- A common type of computation is something like:

```
DO i=1,10                          for (int i=0;i<10;++i) {
  a = a op expr                       a = a op expr
ENDDO                              }
```

- Obviously, there is a loop carried flow dependence for variable 'a' which prohibits parallelization

Reductions

- For these kind of cases OpenMP provides the REDUCTION(op:*list*) clause.
- This clause can be applied when the following restrictions are met
  - *a* is a scalar variable in the list
  - expr is a scalar expression that does not reference *a*
  - Only certain kind of op allowed; e.g. +,*,-
  - For fortran op can also be intrinsic; e.g. MAX,MIN,IOR
  - Vars in *list* have to be shared

Nested Parallelism

- OpenMP allows nested parallelism (e.g. *omp for* within *omp for*)
  - env var OMP_NESTED to enable/disable
  - omp_get_nested(), omp_set_nested() runtime functions
- Compiler can still choose to serialize nested parallel region (i.e. use team with only one thread)
- Can have significant overhead and additional load imbalance

OpenMP Loop collapsing

- When you have perfectly nested loops you can collapse inner loops using the OpenMP clause collapse(n).
  - Loop needs to be perfectly nested
  - Makes iteration space larger
  - Less synchronization needed than nested parallel loops

- Example

```
#pragma parallel for private(i, j) collapse(2)

    for (i =0; i <2; i++)
      for (j =0; j <2; j++)
         foo(A, i, j);
```

UCMERCED

The Worksharing Constructs

```
#pragma omp for
{
    ....
}

!$OMP DO
    ....
!$OMP END DO
```

```
#pragma omp sections
{
        ....
}

!$OMP SECTIONS
        ....
!$OMP END SECTIONS
```

```
#pragma omp single
{
        ....
}

!$OMP SINGLE
        ....
!$OMP END SINGLE
```

- The work is distributed over the threads
- Must be enclosed in a parallel region
- Must be encountered by all threads in team, or none at all
- No implied barrier on entry; implied barrier on exit
- A worksharing construct does not launch any new threads

UCMERCED

More Components of OpenMP

Directives	Runtime Functions	Env. Variables
Array Section Expressions	Number of Threads	Number of Threads
Parallel Region	Thread ID	Scheduling Type
Worksharing Constructs	Dynamic Thread Adjustment	Dynamic Thread Adjustment
SIMD Constructs	Cancellation Status	Thread Affinity Places
Device Constructs	Nested Parallelism	Nested Parallelism
Tasking	Schedlue	Stacksize
Synchronization Constructs	Active Levels	Idle Thread
Cancellation Constructs	Device Selection	Active Levels
Declaration Constructs	Thread Limit	Thread Limit
Memory Flush	Nesting Level	Cancellation
Data-sharing attributes	Ancestor Thread	Default Printout
	Team Size	
	Wallclock Timer	
	Locking	

Red color indicates new addition to OpenMP 4.0

UCMERCED

Potential Speedup

- Ideally, we would like to have perfect speedup (i.e. speedup of N when using N processors)

- This is not realistic in most cases
  - Not all execution time is spent in parallel regions (e.g. loops)
    - e.g. not all loops are parallel (data dependencies)
  - There is an inherent overhead with using openmp threads

TIP: IF Clause

- OpenMP provides another useful clause to decide at run time if a parallel region should actually be run in parallel (multiple threads) or just by the master thread
  - IF (logical expr)

- For example
  - $!OMP PARALLEL IF(n > 100000) (fortran)
  - #pragma omp parallel if (n>100000) (C/C++)

  This will only run the parallel region when n> 100000

Amdahl's Law

- Every program consists of two parts
  - Sequential part
  - Parallel part

- Obviously, no matter how many processors, the sequential part will always be executed by exactly one processor. Suppose, program spends p (0 < p < 1) part of the time in parallel region. Running time (relative to sequential) will be:

$$\frac{(1-p)+\frac{p}{N}}{1}$$

UCMERCED

OpenMP Overhead/Scalability

- Starting a parallel OpenMP region does not come free. There is considerable overhead involved. Consider the following before placing openmp pragmas around a loop
  - Remember Amdahl's law
  - Parallelize most outer loop possible (in some cases even if less iterations)
  - Make sure speedup in parallel region enough to overcome overhead
    - Is the number of iterations in a loop large enough?
    - Is the amount of work per iteration enough?
- Overhead can be different on different machine/OS/compiler

OpenMP Overhead/Scalability

- OpenMP programs don't always scale well, when number of openmp threads increases speedup will suffer
  - More threads competing for available bandwidth
  - Cache issues

- Overhead can be different on different machine/OS/compiler

Reading Materials

- Section 7.10
- OpenMP tutorial from Lawrence Livermore National Lab
  - https://computing.llnl.gov/tutorials/openMP/

Slides references

- Sample slides accompanied with the book “parallel computing”
- Christian Terboven, “Introduction to OpenMP”, July 22, 2013
- Texas A&M University, “OpenMP by Example”, September 17, 2013
- Mary Hall, CS4961 Parallel Programming, University of Utah, 2011
- Ruud van der Pas, OpenMP Tasking Explained, Oracle, November 20, 2013